

Learning Obstacle-Aware Posture Control for a Redundant Robotic Arm

ME5418 Machine Learning in Robotics | Project Group 44

LIN KAIHAO | LIU LINJUN | WANG KEXIN

Revised proposal | 18 September 2026

1. Motivation: why learn posture control?

An inspection robot may keep its tool on a clear path while its elbow strikes nearby equipment. A redundant arm can adopt different joint postures for the same tool position, but a posture that is safe now may leave little room later. The practical problem is therefore to coordinate accurate tool tracking with obstacle-aware motion of the whole arm.

Artificial potential fields provide a conventional approach: nearby obstacles generate repulsive motions [1]. This is interpretable and requires no training, but a reactive rule does not explicitly assess how a posture choice affects the rest of the trajectory. Reinforcement learning (RL) offers a way to learn these sequential trade-offs from repeated episodes, using upcoming reference motion as context. This motivates learning; it does not establish that learning will outperform a well-tuned baseline.

Research question: Can a learned posture controller improve collision-free trajectory tracking compared with a reactive potential-field controller, and under what task conditions? We will retain the same analytical tool-position tracker in both methods and replace only the secondary posture-control component. This focuses the comparison on learned posture decisions rather than learning the entire robot controller.

2. Problem statement and task boundaries

We propose a simulated, fixed-base seven-joint arm in PyBullet [3], initially using the bundled Franka Panda model with its fingers held fixed. The arm will follow a prescribed, time-parameterized 3D tool-position trajectory near one static spherical obstacle. Robot and obstacle geometry are known. The policy may change the arm posture, but not the reference path or its timing. Orientation control, grasping, vision and hardware deployment are outside the project scope.

The minimum task family will use short, smooth line trajectories and varied obstacle placements. Simple cases will establish normal tracking and avoidance behaviour; tighter but feasible layouts will test whether early posture adjustments are useful. Short arcs are a possible extension after the core comparison works. This is continuous articulated-arm control, rather than discrete route planning.

Feasibility and success: each accepted scenario will retain a collision-free joint-motion example that can be replayed under the intended timing, motor limits and tracking tolerance. This checks simulated task feasibility without revealing the example to the policy. Success requires completing the full trajectory without collision or joint-limit violation while remaining within a predefined position-error tolerance throughout. Collision checks include the arm, hand and relevant self-collision pairs. Numerical tolerances and simulation settings will be validated in pilot work and fixed before final comparisons.

3. Controller design and manageable scope

The analytical tracker handles the primary task of following the tool path. A secondary command changes the arm posture using its redundant motion. We initially plan to project that command into directions that minimally affect tool position. The conventional baseline will use obstacle repulsion and joint-limit avoidance; the learned policy will produce bounded joint-posture adjustments through the same interface. Both methods will share the tracker, motor settings and command limits.

This division reduces the learning burden while preserving a meaningful continuous-control problem involving whole-arm geometry and delayed consequences. Projection and command limiting do not guarantee exact tracking or safety, so both must be measured. We will first validate obstacle-free tracking and a working avoidance baseline, then integrate learning. The project contribution is a controlled empirical comparison, not a claim of a new general-purpose RL algorithm.

4. RL formulation and initial learning approach

Observations. The policy will receive joint positions and velocities, current tool-tracking error, desired motion, trajectory progress, upcoming reference positions, obstacle information and the previous executed command. These describe the robot state, present task, future motion and recent control history. We will start with a compact geometric representation and use pilot checks to decide whether explicit link-clearance features are needed; redundant inputs will not be added without justification.

Actions. The policy will output bounded adjustments for the seven arm joints. The shared controller combines these with the analytical tracking command and applies the same physical limits as the baseline. The policy controls posture rather than directly planning a new tool path.

Reward and episode. Each episode follows one complete reference trajectory unless collision or a joint-limit violation ends it early. Rewards will favour accurate tracking, obstacle clearance and smooth commands, penalize unsafe behaviour, and reward successful completion. Breaching the tracking tolerance counts as failure even if a run reaches its time limit. Pilot analysis will check that stopping, oscillating or terminating early cannot become attractive substitutes for completing the task.

Learning method. PPO [2] is our initial choice for bounded continuous control, using small feedforward actor and critic networks. Network size, feature encoding, reward weights and training settings will be selected through development and validation, rather than fixed prematurely. Training will start with simple feasible scenarios and expand only after stable behaviour is observed. The final configuration and model-selection rule will be frozen before held-out evaluation.

5. Evaluation: what would constitute useful evidence?

The core comparison will include learned posture control, a tuned potential-field posture controller, and a tracker without secondary avoidance. Each will run the same test trajectories and obstacles from identical initial states, with common sensing availability, limits and collision checks. Training, validation and test scenarios will be separated; baseline tuning and checkpoint selection will use validation data only.

The primary outcome is full-trajectory collision-free tracking success. Supporting measures will include collision rate, tracking error, minimum obstacle clearance, command smoothness, velocity saturation and online decision time. Early failures will be reported with their failure reason and completed duration; short failed runs will not be pooled with full runs to claim better tracking or smoothness. Continuous metrics on jointly completed scenarios will be labelled as conditional comparisons, with the number of such scenarios shown.

We aim for three independent training seeds and at least 100 fixed held-out scenarios per policy, covering both simple and tighter layouts. Results will show variation across seeds and task conditions, rather than only an overall average. If resources permit, a separately trained policy without future-reference inputs will test their contribution. Without that ablation, any improvement will be attributed to the overall learned controller. A strong conventional baseline or a negative RL result remains a useful answer to the research question.

6. Implementation plan, risks and deliverables

Stage 1: validate the robot model, obstacle-free tracking, collision checks and feasible task examples. **Stage 2:** implement and tune the conventional baselines and complete the common evaluation pipeline. **Stage 3:** run a small PPO pilot to establish learning behaviour, simulation throughput and a realistic compute budget, then train the final models. **Stage 4:** evaluate frozen models, inspect failure cases and prepare reproducible comparisons.

The main risks are simulator integration, uninformative task difficulty and training cost. If time is limited, we will retain one obstacle, line trajectories and the three core controllers, dropping extra trajectory families and optional ablations first. Any reduced training or evaluation scope will be declared. Deliverables will include the environment, controller and learning code, saved models, configurations, evaluation results and comparison videos. We will document reused code/assets and maintain the commitment to at least 50% original project code.

References and AI-use declaration

- [1] O. Khatib. Real-Time Obstacle Avoidance for Manipulators and Mobile Robots. IJRR, 5(1), 90-98, 1986. doi:10.1177/027836498600500106.
- [2] J. Schulman et al. Proximal Policy Optimization Algorithms. 2017. arXiv:1707.06347.
- [3] Bullet Physics. PyBullet Quickstart Guide and bundled franka_panda/panda.urdf. github.com/bulletphysics/bullet3 (accessed 18 September 2026).

AI-use declaration: OpenAI Codex assisted with proposal formulation, peer-feedback analysis, source checking, drafting and formatting. The team is responsible for the submitted content, implementation and verification. No project experiments are claimed here.